

Just make it

我们需要：编译所有的源代码，把目标文件链接起来，生成elf文件，生成bin硬盘镜像，用qemu跑起来

这一系列复杂的命令，我们不想每次用到的时候都敲一遍，所以我们使用魔改的祖传 Makefile 。

make 和 Makefile

GNU make(简称make)是一种代码维护工具，在大中型项目中，它将根据程序各个模块的更新情况，自动的维护和生成目标代码。

make命令执行时，需要一个 makefile （或Makefile）文件，以告诉make命令需要怎么样的去编译和链接程序

我们的 Makefile 还依赖 tools/function.mk 。只要我们的makefile写得够好，所有的这一切，我们只用一个make命令就可以完成，make命令会自动智能地根据当前的文件修改的情况来确定哪些文件需要重编译，从而自己编译所需要的文件和链接目标程序。

makefile的基本规则简介

在使用这个makefile之前，还是让我们先来粗略地看一看makefile的规则。

```
target ... : prerequisites ...  
    command  
    ...  
    ...
```

target也就是一个目标文件，可以是object file，也可以是执行文件。还可以是一个标签（label）。prerequisites就是，要生成那个target所需要的文件或是目标。command也就是make需要执行的命令（任意的shell命令）。这是一个文件的依赖关系，也就是说，target这一个或多个的目标文件依赖于prerequisites中的文件，其生成规则定义在 command中。如果prerequisites中有一个以上的文件比target文件要新，那么command所定义的命令就会被执行。这就是makefile的规则。也就是makefile中最核心的内容

Runing ucore

在源代码的根目录下 make qemu ，在makefile中运行的代码为

```
.PHONY: qemu
qemu: $(UCOREIMG) $(SWAPIMG) $(SFSIMG)
# $(V)$(QEMU) -kernel $(UCOREIMG) -nographic
$(V)$(QEMU) \
    -machine virt \
    -nographic \
    -bios default \
    -device loader,file=$(UCOREIMG),addr=0x80200000
```

这段代码就是我们启动qemu的命令，这段代码首先通过宏定义\$(UCOREIMG) \$(SWAPIMG) \$(SFSIMG)的函数进行目标文件的构建，然后使用qemu语句进行qemu启动加载内核。我们就把ucore跑起来了，运行结果如下。

```
+ cc kern/init/init.c
+ cc libs/sbi.c
+ ld bin/kernel
riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img
```

OpenSBI v0.6

```

      _____
     /  _  \           /  _  |  _  \  _  |
    |  |  |  _  _   _  | (  _  |  _  |  |  | | | |
    |  |  |  '  \ /  _  \ '  \  \  _  \ |  _  <  |  |
    |  |  |  |  _  |  _  /  |  |  _  )  |  _  |  |  |
    \  _  /  |  _  \  _  |  |  |  _  /  |  _  /  _  |
      |  |
      |  |

```

```
Platform Name       : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs   : 8
Current Hart        : 0
Firmware Base       : 0x80000000
Firmware Size       : 120 KB
Runtime SBI Version  : 0.2
```

```
MIDELEG : 0x00000000000000222
MEDELEG : 0x0000000000000b109
PMP0    : 0x0000000080000000-0x000000008001ffff (A)
PMP1    : 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
(THU.CST) os is loading ...
```

它输出一行 (THU.CST) os is loading , 然后进入死循环。

tips:

- 关于Makefile的语法, 如果不熟悉, 可以查看GNU手册, 或者这份中文教程:
<https://seisman.github.io/how-to-write-makefile/overview.html>
- 对实验中Makefile感兴趣的同学可以阅读附录中的Makefile